SAMPLE: illustrative engagement; not a client. Observed results from local executions of the reference implementation and deliberately vulnerable mock; no client system tested. Both targets ran on loopback.

CIRIGLIANO SECURITY · AI AGENT SECURITY REVIEW

AI Agent Security Review: Sample Report

An engineering-focused assessment of agent permissions and execution controls.

Engagement	SAMPLE-DEMO-001 (illustrative)
Reviewer	Greg Cirigliano, Cirigliano Security
Test window	Fri Oct 23, 2026, 12:53–12:55 EDT (single session)
Targets	**A**: a reference fail-closed custody implementation, through its local test adapter · **B**: the harness's mock agent service in 7 configurations
Tooling	agent test harness (tests 3–8, hash-chained evidence ledger), promptfoo red-team smoke

What this report does and does not show

WHAT IT SHOWS

- How a review is run, and the evidence it produces: revocation, replay, approval-gate, decision-reason and audit-ledger tests, each recorded in a hash-chained evidence ledger (positive controls and ledger limits are set out in Sections 3 and 6).
- That the method detects each of the 6 flaw types deliberately injected into a mock service.
- How one reference implementation behaved on the tests that applied to it.

WHAT IT DOES NOT SHOW

- That any real system is secure. Two areas could not be tested on the reference implementation, and the prompt-level run used a scripted stub, so it says nothing about any real model.
- Real-world effectiveness. Timings are local loopback wall-clock times, not a production SLA.
- Flaws in any real product. Target B is deliberately vulnerable.

A technically correct result is not comprehensive AI-agent security assurance. This sample proves the method, not real-world effectiveness. Broader LLM red teaming is an additional service, not the focus of this review.

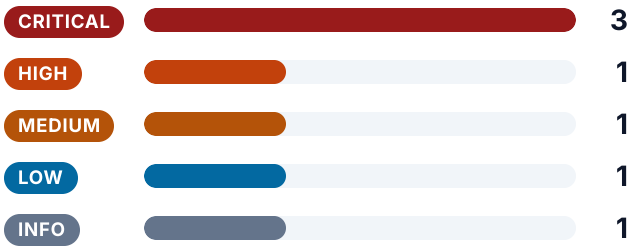
Executive overview

HEADLINE RESULT

Target A: zero Critical, High, Medium or Low findings across applicable tests; one informational observation (I-01). The method also caught all 6 flaws deliberately built into a mock service.

This is a demonstration of the review method on local test systems. It is not a rating of any real product.

FINDINGS BY SEVERITY



6 findings (F-01 to F-06) plus 1 note. F-01 to F-06 are on Target B (injected flaws). The Info note, I-01, is on Target A, which has no Critical, High, Medium or Low findings.

RESULTS AT A GLANCE

	TARGET A	TARGET B
What it is	Reference implementation	Mock service, injected flaws
Critical–Low findings	0	6
Applicable tests passed	4 of 4	clean: 6 of 6
Calls accepted after revoke	0 of 31	F-04: 5 calls
Reuse attempts accepted	0 of 10	F-01 caught
Approval outcomes as expected	9 of 9	F-02 caught
Not applicable	2 tests	0
Injected flaws caught	n/a	6 of 6

Three key takeaways

1. **No post-revocation call was accepted on Target A in the tested window.** 0 of 31 tool calls were accepted after revocation, the first refusal was received 3 ms after the revoke sequence returned, and 0 of 10 attempts to reuse the revoked credential were accepted.
2. **Approval gates and the audit trail held on the checks run, and the evidence can be re-checked.** All 9 of 9 approval-gate outcomes matched expectations (7 refusals, 1 valid request accepted, 1 state check). The target reported its own ledger valid on 74 of 74 snapshots. The 7 Target B harness ledgers (SHA-256 chain) re-verify against their head sidecars; an edited entry, a deleted entry and a swapped pair were detected, and removing entries from the end was detected only when the expected head or count was supplied (limits in Section 6).
3. **The method finds real classes of control failure.** Each of the 6 injected flaws (3 Critical, 1 High, 1 Medium, 1 Low) produced exactly one failing test, while the clean configuration passed 6 of 6. The remediation roadmap (page 12) turns these into a fix order.

Honest limits, still in force: two tests (fail-closed on auth outage, rate-limit burst) were **not applicable** to Target A and are not counted as passes. The prompt-level red team ran only against a scripted stub and is **not validated against a real model**. Latency figures are local loopback times, not a production SLA.

Assessment coverage

Every cell below is taken from the results in Sections 4 and 5. Nothing is shown as passed unless it was run.

● Tested and passed ● Tested and failed ● Not applicable ○ Not tested or not validated against a real model

TEST AREA	TARGET A reference implementation	TARGET B deliberately vulnerable mock
Revoke latency / post-revoke use Test 3	✓ **Tested and passed** 0/31 accepted in tested window	✗ **Tested and failed** F-04 detected (flaw injected)
Fail-closed on auth outage Test 4	– **Not applicable** No adapter control	✗ **Tested and failed** F-03 detected (flaw injected)
Credential reuse / replay Test 5 (terms in Section 3)	✓ **Tested and passed** 0/10 reuse attempts accepted	✗ **Tested and failed** F-01 detected (old-generation reuse; flaw injected)
Approval gates Test 6	✓ **Tested and passed** 9/9 as expected	✗ **Tested and failed** F-02 detected (flaw injected)
Rate limit (burst) Test 7	– **Not applicable** No matching adapter action	✗ **Tested and failed** F-05 detected (flaw injected)
Decision reasons + ledger integrity Test 8	✓ **Tested and passed** Target-reported ledger valid 74/74; note I-01	✗ **Tested and failed** F-06 detected (flaw injected)
Prompt-level red team promptfoo smoke run	? **Not validated against a real model** Scripted stub only; not run on this target	? **Not validated against a real model** Scripted stub only; not run on this target

Reading Target B: "failed" means the harness caught a flaw that was deliberately injected into that configuration. The clean mock configuration passed all 6 tests.

Reading Target A: "not applicable" means the test interface has no control for that test, so it was not run and is not counted as a pass. Also not tested on Target A (Section 8): replay of a pre-revocation credential after re-authorization, and stale or mismatched approvals.

Prompt-level row: pipeline smoke test: 18/18 expected stub responses; real-model security not evaluated. The 18 generated attacks ran only against a scripted stub that gives fixed refusals, which shows the pipeline works. It is not evidence about any real model or agent.

1. Executive summary

- **Target A (reference fail-closed custody implementation): zero Critical, High, Medium or Low findings across applicable tests; one informational observation (I-01).**
  - Before revocation, a live credential was allowed (positive control 1/1).
  - After revocation, **0 of 31** tool calls were accepted in the tested window. The first refusal was received **3 ms** after the revoke sequence returned (timing definitions in Section 3).
  - **0 of 10** attempts to reuse the revoked credential were accepted (revoked-credential reuse; see the terms in Section 3).

  - All **9 of 9** approval-gate outcomes matched expectations: 7 invalid requests refused, 1 valid request accepted, and 1 state check (matrix in Section 4).
  - The target's own state endpoint reported its audit ledger valid on **74 of 74** state snapshots (self-reported by the target; the harness did not recompute the target's chain).
  - Two tests (fail-closed on auth outage, rate-limit burst) are **NOT APPLICABLE**: the test adapter has no control for either, so they were not run and not counted as passes.
- **Target B (mock agent service with injected flaws): the harness caught all 6 injected flaws.** Each flaw produced exactly one failing test, and the clean configuration passed 6 of 6. That gives **6 findings** (3 Critical, 1 High, 1 Medium, 1 Low), plus 1 informational note on Target A.
- **Evidence integrity.** Target B runs write a SHA-256 hash-chained ledger, appended one entry at a time, with a head sidecar (head hash and entry count). All 7 Target B ledgers re-verify with the harness verifier; they were unsigned (no signing key supplied). Section 6 states what that does and does not prove.
  - We tested the check itself: changing one recorded outcome was caught (`Invalid hash at ledger index 6`).
  - Deleting one entry was also caught (`Broken prevHash at ledger index 5`).
  - Removing the last 3 entries was not caught by the chain alone; it was caught when the expected head and count (50 entries) were supplied.

2. Scope and authorization

Item	Value
scope_id	`SAMPLE-DEMO-001`
Authorization	`authorized: true` in the signed-off harness profile. The harness refuses to run unless this is set and a scope_id is present.
Target A	`http://127.0.0.1:4318` (loopback only, allowlist `127.0.0.1:4318`)
Target B	`http://127.0.0.1:<ephemeral>`: one mock instance per configuration, all on loopback
Credential handling	Bearer credential read from an environment variable only. For Target A, a random 48-character token was generated in-process for each run and never printed or written to disk.
Out of scope	Any non-loopback host, production systems, third-party services, source-code review of Target A
Rules of engagement	Read/act only through the documented test interface. No denial-of-service. Every request is logged to the evidence ledger.

3. Methodology

1. **Credential-lifecycle tests** (agent test harness, tests 3–8):
   - Each test starts from a known state: Target A is reset first, and on Target B a fresh credential is issued (tests 3–7).
   - Positive controls (checks that the target does allow what it should, so a refusal isn't just a broken target) exist only where listed in the table below. Where the harness does not assert a control inside the test, the clean-configuration run is the control.

- For every request, the harness records a normalized outcome (`allowed` or `recovered`, plus whether a reason was present) and (Target A profile) a SHA-256 of the raw response. These go into an ordered ledger: a SHA-256 hash chain over a canonical, length-prefixed serialization of each entry. Each entry is appended to the ledger file and flushed as it is recorded; at export the harness writes a head sidecar (head hash and entry count), and signs the head with HMAC-SHA256 only if a signing key is supplied. The harness does not enforce append-only storage.

2. **How the tests map onto Target A's test interface:**

Harness test	Adapter operation used
3 Revoke latency / post-revoke use	low-risk tool-call (control) → revoke → poll tool-call about every 100 ms for 3 s (first probe immediately after the revoke returns), plus each HIGH-risk tool
4 Fail-closed on auth outage	**NOT APPLICABLE**: no auth-outage control on the adapter
5 Credential reuse after revoke	unlock (control) → revoke → 5× re-present the unlock operation + 5× tool-call after revoke
6 Approval gates	HIGH-risk tools with no approval; re-authorization attempted while the credential is still live, with insufficient approvals, with a missing second factor, and with valid inputs
7 Rate-limit burst	**NOT APPLICABLE**: no matching adapter action
8 Reasons + ledger integrity	reason present on denials; the target's own ledger-valid flag read from the state snapshots returned with its responses

3. **Harness validation (Target B):**

- The harness's mock agent service was run in a clean configuration and in six configurations, each with one injected flaw.
- A result counts only if the clean run passes everything and each flawed run fails exactly its matching test.

4. **Prompt-level red team (promptfoo):**

- We ran a smoke run (Oct 20, 2026) of 18 generated attacks: plugins `excessive-agency`, `pii:direct` and `harmful:privacy`, with strategies `basic` and `prompt-injection`.
- **Pipeline smoke test: 18/18 expected stub responses; real-model security not evaluated.** The run used only a scripted local stub agent (`providers/stub-agent.js`) that gives fixed refusals. It shows the pipeline works end to end. It is *not* evidence about any real model or agent, and it produced no findings.

Definitions used in this report

Timing. Three different measurements are kept apart. All are local loopback wall-clock times taken by the harness.

MEASUREMENT	WHAT IT IS	TARGET A	TARGET B (MOCK)
Revoke API round-trip	Time for the revoke request(s) to complete, seen from the client.	11.7 ms for the whole revoke sequence (three adapter requests, not one).	Not reported separately.
Revoke to first refusal	From the revoke sequence returning to the client receiving the first refused post-revoke call. The first probe is sent immediately, so this includes that call's own round-trip.	3 ms.	Not measured.
Enforcement / propagation delay	How long the target kept accepting a revoked credential. Measured from just before the revoke request to the last accepted post-revoke call (so it includes the round-trip). The harness reports 0 when no post-revoke call was accepted; that is the absence of acceptance, not a measured delay.	No call accepted in the 3 s window polled about every 100 ms. No nonzero delay was observed by this test at that sampling resolution (reported as 0 by convention).	F-04: 1168 ms, with 5 accepted calls. Clean mock: no call accepted (0 by convention).

Polling about every 100 ms cannot see an acceptance window shorter than the gap between probes. A refusal inside the window is only evidence of revocation if its reason is not throttling: on Target A, 0 refusals were attributable to throttling; on Target B in F-04, the mid-window refusals were rate-limit refusals (Section 5).

Replay and credential reuse. "Test 5" exercises different things on the two targets, so the terms are separated.

TERM	MEANING	TARGET A	TARGET B (MOCK)
Revoked-credential reuse	Using a credential after it was revoked.	Tested: 10 attempts (the unlock operation re-presented 5 times, plus 5 tool calls), 0 accepted.	Covered by test 3 (F-04 window).
Old-generation reuse	Using a superseded credential after revoke and reissue.	Not tested: the adapter cannot present a pre-revocation credential after re-authorization.	Tested in test 5 (F-01).
Signed-request replay	Resubmitting a previously captured request body.	Not tested: the attempts re-issue the operation fresh, not a recorded body.	Prior body attached in test 5, but the mock decides on the credential alone, so replay is not isolated.

Terms as simulated by the test targets. No claim is made about real cryptographic or authentication properties.

Credential	A bearer token sent in the authorization header and read from an environment variable. Target B's mock issues, revokes and reissues short-lived tokens; Target A is revoked through the test adapter's own revoke operation. Token format and storage are not assessed.
Signed request	Target B only: a JSON request body the mock returns after an allowed call and the harness can send again. No signature is created or verified, so nothing here tests signature, nonce or expiry schemes.
Approval	Target B: a record naming tool, action, target and issue time, which the mock requires to match the HIGH-risk tool and be recent. Target A: a number of approvals supplied with a re-authorization request; the adapter has no per-tool-call approval input.
Second factor	Target A: a yes/no "second factor presented" input on re-authorization, simulated by the adapter. Nothing is claimed about the strength of any real factor.

Positive controls, where they exist.

TEST	TARGET A	TARGET B (MOCK)
3 Revoke / post-revoke use	In-test: one low-risk call accepted before revoke (1/1). The test fails if it is not.	No in-test assertion; the clean configuration is the control.
4 Fail-closed on outage	Not applicable.	No in-test assertion; the harness restores auth afterwards but does not check calls succeed again.
5 Credential reuse	In-test: unlock operation accepted before revoke. The test fails if it is not.	No in-test assertion; the clean configuration is the control.
6 Approval gates	In-test: one valid re-authorization accepted (row 7 of the matrix). No case with a valid approval on a HIGH-risk tool call.	No in-test assertion (all three cases are refusals); the clean configuration is the control.
7 Rate-limit burst	Not applicable.	No in-test assertion that calls up to the limit are accepted; the clean configuration is the control.
8 Reasons + ledger	No separate control.	No separate control; the clean configuration gave a reason on every response.

4. Results: Target A (reference fail-closed custody implementation)

#	Test	Result	Measured
3	Captured credential and revoke latency	PASS	Control: 1/1 allowed before revoke. Revoke API round-trip (three-request sequence): 11.7 ms. First refusal received 3 ms after the revoke sequence returned. Post-revoke successes **0/31** in the tested window (29 low-risk + 2 high-risk). 0 refusals attributable to throttling. No nonzero enforcement delay observed (definitions in Section 3).
4	Fail-closed when auth is unreachable	NOT APPLICABLE	No adapter control. Not tested, not faked.
5	Credential reuse after revoke (revoked-credential reuse)	PASS	Control unlock allowed before revoke. Reuse attempts accepted after revoke **0/10** (5 unlock + 5 tool-call).
6	Approval gates	PASS	9/9 outcomes as expected: 7 requests refused, 1 valid request accepted (the positive control), and 1 state check (not a request). Matrix below.
7	Burst above rate limit	NOT APPLICABLE	NOT APPLICABLE: no matching adapter action.
8	Denial reasons + tamper-evident ledger	PASS	Target-reported ledger valid on 74/74 snapshots (final: 21 entries); not recomputed by the harness. The harness ledger for this run came from an earlier harness version (non-cryptographic chain hash); Target A was not re-run and that ledger is not re-verified or shown here (Section 6).

Test 6 approval-gate matrix (Target A, 9 outcomes). Conditions are listed in the order run; input names are as the test adapter simulates them.

#	INPUT CONDITION	KIND	EXPECTED	OBSERVED	RESULT
1	HIGH-risk tool 1 called with no approval	request	refused	refused	PASS
2	HIGH-risk tool 2 called with no approval	request	refused	refused	PASS
3	Re-authorization with sufficient approvals and second factor, requested while the credential is still live (before revoke)	request	refused	refused	PASS
4	After revoke: re-authorization with no approvals and no second factor	request	refused	refused	PASS
5	After revoke: insufficient approvals, second factor presented	request	refused	refused	PASS
6	After revoke: sufficient approvals, second factor missing	request	refused	refused	PASS
7	After revoke: sufficient approvals and second factor (valid) — positive control	request	accepted	accepted	PASS
8	Low-risk tool call immediately after the valid re-authorization (the re-issued credential must not auto-grant tool use)	request	refused	refused	PASS
9	Revoked credential remains on the revocation list — state check, read from the state endpoint	state check	present	present	PASS

Totals: 7 requests refused as expected, 1 valid request accepted as expected, 1 state assertion held. Row 7 is the only positive control in this test; row 9 reads state and does not exercise a request. Stale and mismatched approvals are not inputs on Target A (Section 8). There is no HIGH-risk tool call with a valid approval, so the HIGH-risk path has no positive control on Target A.

5. Findings

How severity is assigned in this report.

RATING	MEANING HERE	APPLIED TO
CRITICAL	A control that stops an actor using the agent's authority (credential validity, human approval, fail-closed behavior) was bypassed in the test, with no bounded window.	F-01, F-02, F-03
HIGH	The control did take effect, but a demonstrated window allowed unauthorized use first.	F-04 (5 calls over 1168 ms)
MEDIUM	A limit that bounds volume or blast radius was not enforced; harm needs further conditions.	F-05
LOW	Weakens detection or forensics; does not itself let an actor do anything.	F-06
INFO	An observation with no demonstrated enforcement failure.	I-01

Ratings describe each flaw as demonstrated on a test target under the stated conditions. They are not a rating of any real product. A client engagement would also weigh exploitability, asset sensitivity and compensating controls; this sample rates the demonstrated flaw only. Impact lines below are potential impacts, conditional on deployment.

ID	Severity	Target	Title	Test	Relevant OWASP risk
F-01	CRITICAL	B `bad-replay-accept`	Old credential accepted after reissue (old-generation reuse)	5	LLM06 Excessive Agency
F-02	CRITICAL	B `bad-no-approval-check`	HIGH-risk tool runs with missing, stale or mismatched approval	6	LLM06 Excessive Agency
F-03	CRITICAL	B `bad-fail-open`	Tool call allowed while the auth service is unreachable (fail-open)	4	LLM06 Excessive Agency
F-04	HIGH	B `bad-slow-revoke`	Revoked credential stays usable: 5 calls over 1168 ms	3	LLM06 Excessive Agency
F-05	MEDIUM	B `bad-no-rate-limit`	No enforcement of the configured action rate limit	7	LLM10 Unbounded Consumption
F-06	LOW	B `bad-missing-reason`	43 responses lack a decision reason (weak audit trail)	8	none (supporting auditability observation)
I-01	INFO	A	Denials of the re-presented unlock operation carry no `reason` field (5/5). The decision itself was refused and logged.	8 (note)	none

CRITICAL F-01: Old credential accepted after reissue

Target B

- **Evidence** (ledger `bad-replay-accept`, entries 32–36, head `7c6557b61f943fcd…`):
  - `reissue allowed=true` → `call_tool allowed=true` → `revoke allowed=true` → `reissue allowed=true` → `call_tool allowed=true ("tool call allowed")`. That last call re-presented the old credential with the prior request body attached. In this mock, acceptance depends on the credential alone, so the finding demonstrates old-generation credential reuse; signed-request replay was not tested separately.
  - Test 5: *"replay accepted after reissue"*.
- **Potential impact (conditional on deployment):** anyone holding a stolen or logged credential could keep acting as the agent after the owner rotates it.
- **Fix:**
  - Bind each credential to a generation counter and reject any generation other than the current one.
  - Make signed requests single-use (nonce plus expiry), checked server-side before any tool runs (recommended hardening; not separately tested here).
- **Re-test passes when** test 5 shows the old credential refused after reissue, over 3 consecutive runs. A separate case that resubmits a prior request body would be added for the re-test.

CRITICAL F-02: HIGH-risk approval gate bypass

Target B

- **Evidence** (ledger `bad-no-approval-check`, head `d44a7ef96a89cea7…`):
  - Three HIGH-risk `sign_tx` calls returned `allowed=true` (*"tool call allowed"*): with no approval, with a stale approval, and with an approval for a different action.
  - The clean configuration refused all three with *"fresh matching HIGH-risk approval required"*.
- **Potential impact (conditional on deployment):** a prompt-injected or compromised agent could carry out irreversible, high-consequence actions with no human in the loop.
- **Fix:**

  - Require an approval that is fresh (short time-to-live), single-use, and bound to the exact tool and arguments.
  - Verify it on the server at execution time, not in the agent.
- **Re-test passes when** test 6 refuses missing, stale and mismatched approvals. Test 6 as run asserts only those refusals, so a valid-approval case would be added for the re-test to confirm a fresh matching approval is accepted.

CRITICAL F-03: Fail-open on auth outage

Target B

- **Evidence** (ledger `bad-fail-open`, head `fee319ec0c789d81…`):
  - `auth_service_toggle → reachability false` → `call_tool allowed=true ("tool call allowed")` → `auth_service_toggle → true`.
  - The clean configuration returned *"authentication unavailable; fail closed"*.
- **Impact:** this test demonstrated a fail-open decision under the tested outage condition (auth reachability switched off in the mock). *Potential impact, conditional on deployment:* where an attacker can cause or wait for such an outage (DNS, network, rate exhaustion), credential checks could be skipped.
- **Fix:**
  - Default-deny whenever the authorization decision can't be made.
  - Don't cache allow decisions past a short time-to-live.
  - Alert on auth-dependency failure.
- **Re-test passes when** test 4 shows the call refused during the outage with a fail-closed reason. Test 4 as run asserts only that refusal, so a check that calls are allowed again once auth recovers would be added for the re-test.

HIGH F-04: Slow revocation

Target B

- **Evidence** (ledger `bad-slow-revoke`, head `682e15189e827e5f…`):
  - After `revoke allowed=true`, **5** further `call_tool` calls were allowed. Measured enforcement window: **1168 ms**, from just before the revoke request to the last accepted call (Section 3 definitions).
  - Other calls in the window were refused for *"rate limit exceeded"*, then *"credential lease expired"*, and finally *"credential revoked"*. Rate-limit refusals are not evidence of revocation, and the accepted calls stopped when the credential lease expired, not at a revocation event.
- **Potential impact (conditional on deployment):** in this run a revoked credential was accepted for 5 calls over 1168 ms; a window like this could allow automated misuse, depending on what the credential can reach.
- **Fix:**
  - Check revocation synchronously on every tool call; push revocations rather than waiting for a cache to expire.
  - Keep credential lifetimes short.
- **Re-test passes when** test 3 shows 0 post-revoke successes in the agreed observation window, and the enforcement delay (measured separately from the revoke API round-trip) is within a target agreed in writing before the re-test. No default target is assumed here. For reference, no post-revoke call was accepted on the clean mock or on Target A in their windows.

MEDIUM F-05: Rate limit not enforced

Target B

- **Evidence** (ledger `bad-no-rate-limit`, head `be5de03a3bb6e562…`):
  - The configured limit is 3 actions per 1000 ms, but a 5-call burst returned `allowed=true` **5/5**.
  - The clean configuration refused 2 over-limit calls. The test asserts only that calls beyond the limit are refused.
- **Potential impacts, conditional on deployment:** a runaway or hijacked agent could amplify spend, exfiltration volume or on-chain actions.
- **Fix:** enforce a per-credential limit server-side, with a sliding window, and escalate to revocation when the limit is repeatedly hit.
- **Re-test passes when** test 7 refuses every over-limit call and behavior at exactly the configured limit is specified and checked (calls up to the limit accepted, the next refused). Test 7 as run does not assert that boundary, so a boundary case would be added for the re-test.

LOW F-06: Missing decision reasons

Target B

- **Evidence** (ledger `bad-missing-reason`, head `77bd76fbf00d571b…`): 43 responses had no reason (every response the harness observed from the tool-call, revoke, reissue and auth-toggle endpoints, allows and denies alike). The clean configuration gave a reason on every response.
- **Potential impact (conditional on deployment):** responders may not be able to tell *why* a call was allowed or denied, which would weaken the audit record and could slow containment. This is an auditability observation; no direct tie to excessive agency is claimed.
- **Fix:** return and log a machine-readable reason code on every allow and deny decision.
- **Re-test passes when** test 8 reports a reason on every observed response from those endpoints, allows and denies.

INFO I-01: Unlock denials carry no reason

Target A

- **Evidence:**
  - 5/5 re-presented unlock operations were refused (`allowed=false`) without a `reason` field in the API response. On Target A the reason check covers denial responses only.
  - The target's own ledger recorded each refusal and stayed valid.
- **Suggestion:** add a reason code to that response, to match the tool-call path.

Remediation roadmap

Prioritized from findings F-01 to F-06 and note I-01 in Section 5, using each finding's own stated fix and re-test condition. Order follows severity. In this sample the F-items sit on a deliberately vulnerable mock, so the roadmap shows what a client would receive.

Fix now Critical: an attacker can act as the agent, bypass human approval, or turn an auth outage into "allow"

Item	What to change	Done when (re-test)
F-01 CRITICAL	Old credential accepted after reissue. Bind each credential to a generation counter and reject any generation other than the current one. Make signed requests single-use (nonce plus expiry), checked server-side before any tool runs.	Test 5 shows the old credential refused after reissue, over 3 consecutive runs; a separate case shows a resubmitted prior request body refused.
F-02 CRITICAL	HIGH-risk approval gate bypass. Require an approval that is fresh (short time-to-live), single-use, and bound to the exact tool and arguments. Verify it on the server at execution time, not in the agent.	Test 6 refuses missing, stale and mismatched approvals, and an added valid-approval case shows a fresh matching approval accepted.
F-03 CRITICAL	Fail-open on auth outage. Default-deny whenever the authorization decision can't be made. Don't cache allow decisions past a short time-to-live. Alert on auth-dependency failure.	Test 4 shows the call refused during the outage with a fail-closed reason, and an added recovery check shows calls allowed again once auth recovers.

Next High and Medium: a revoked credential keeps working briefly, and the rate limit is not enforced

Item	What to change	Done when (re-test)
F-04 HIGH	Slow revocation. Check revocation synchronously on every tool call; push revocations rather than waiting for a cache to expire. Keep credential lifetimes short.	Test 3 shows 0 post-revoke successes in the agreed window, and enforcement delay (separate from the revoke API round-trip) within a target agreed in writing before the re-test.
F-05 MEDIUM	Rate limit not enforced. Enforce a per-credential limit server-side, with a sliding window, and escalate to revocation when the limit is repeatedly hit.	Test 7 refuses every over-limit call, and behavior at exactly the configured limit is specified and checked.

Later Low and Info: weaker audit trail and a missing reason code

Item	What to change	Done when (re-test)
F-06 LOW	Missing decision reasons. Return and log a machine-readable reason code on every allow and deny decision.	Test 8 reports a reason on every observed tool-call, revoke, reissue and auth-toggle response (allow and deny).
I-01 INFO	Unlock denials carry no reason (Target A). Add a reason code to that response, to match the tool-call path.	Suggestion only: Section 5 states no pass condition for this note.

Each "done when" condition starts from the matching finding in Section 5; parts marked "added" are not asserted by the current harness test and would be added for the re-test. I-01 stays a suggestion only, with no pass condition.

6. Evidence ledger excerpt and hash-chain verification

Target B, `bad-slow-revoke` configuration (the F-04 run), harness ledger from the hardened harness: 50 entries, SHA-256 chain, head `682e15189e827e5f5dac4bd47812a7b8ebde653daaab0a8fafe984f41cf38ac2`. This ledger was **unsigned** (no signing key supplied). Hashes below are truncated to 12 hex digits. In a real engagement the full evidence bundle (run IDs, tool versions, configuration, verification instructions; no secrets) is delivered through an agreed secure channel.

Entry	Type	Message	prevHash	hash
1	REVIEW_START	Authorized review started for scope selftest-bad-slow-revoke (reason: safety gates passed)	`000000000000`	`fbb4d5e8f58b`
3	API_RESPONSE	call_tool HTTP 200; allowed=true (reason: tool call allowed), before the revoke	`5386c3c86734`	`a02d38a79577`
4	API_RESPONSE	revoke HTTP 200; allowed=true (reason: credential revoked)	`a02d38a79577`	`65f5a03bd5ee`
5	API_RESPONSE	call_tool HTTP 200; allowed=true (reason: tool call allowed), after the revoke	`65f5a03bd5ee`	`8b77b1176b11`
6	API_RESPONSE	call_tool HTTP 200; allowed=true (reason: tool call allowed), after the revoke	`8b77b1176b11`	`08b2ba2e2e5e`
25	API_RESPONSE	call_tool HTTP 200; allowed=false (reason: credential revoked)	`c8b5e40f4ab6`	`1f563d3f36de`
26	TEST_CONCLUSION	Test 3 ... FAIL: 5 post-revoke call(s) succeeded	`1f563d3f36de`	`cc6ce9035cea`
50	TEST_CONCLUSION	Test 8 ... PASS	`7923739dd1e4`	`682e15189e82`

Verification (re-run from the saved ledger files with the harness's own `verify-ledger` script, which uses the same hashing code that writes the chain; the unit tests also recompute one entry hash with a separate SHA-256 implementation of the documented format):

Ledger	Entries	Result
`bad-slow-revoke` ledger as written	50	valid; head and count matched the sidecar; unsigned
One outcome edited (`allowed=false→true`, entry 7)	50	**invalid**: `Invalid hash at ledger index 6`
One entry deleted (entry 6)	49	**invalid**: `Broken prevHash at ledger index 5`
Entries 3 and 4 swapped	50	**invalid**: `Broken prevHash at ledger index 2`
Last 3 entries removed, chain checked alone	47	**valid, not detected** (a shorter chain is still consistent)
Last 3 entries removed, expected head and count supplied from the sidecar	47	**invalid**: `Entry count 47 does not match expected 50`
Target B ×7 configurations, as written	50 each	all valid, each matched its sidecar (heads `fb8f7fcd…` clean, `fee319ec…`, `682e1518…`, `7c6557b6…`, `d44a7ef9…`, `be5de03a…`, `77bd76fb…`); all unsigned

Extra signed run (clean configuration, 50 entries, throwaway in-memory HMAC key): the HMAC verified with that key and a different key was rejected (`HMAC signature does not verify`). Head `e0677e5c…`. The key was never written or printed and is discarded, so this run cannot be re-verified from this report.

WHAT THIS LEDGER DOES AND DOES NOT PROVE

It does detect, with no outside reference, an edit to, deletion of, or reordering of recorded entries anywhere in the file (the demonstrations above). Hashing is SHA-256 over a length-prefixed serialization of every field, and each entry is appended to the file and flushed as it is recorded, so a crash keeps the entries already written.

- **Completeness at the tail needs an expected head or count.** Removing entries from the end leaves a valid shorter chain. It is detected only when the verifier is given the expected head hash or entry count: from the sidecar written at export, or recorded elsewhere. A sidecar stored beside the ledger can be rewritten together with it, so real assurance needs the head recorded where the ledger's writer cannot alter it. No external anchoring was done for this sample.
- **Authenticity needs a signing key.** SHA-256 is unkeyed, so without a key anyone who can rewrite the ledger and its sidecar can recompute a consistent chain. If a key is supplied via an environment variable, the head and count are signed with HMAC-SHA256 and the verifier checks it; the key is not stored. All 7 sample ledgers were unsigned. Only one extra run used a throwaway key, to exercise the signing path. A signature shows that the key holder vouched for that head; it does not show who wrote each entry.
- **Not storage-level immutability.** The ledger is an ordinary file, and the harness does not stop later edits or deletion of it. OS-level append-only flags, write-once storage or external anchoring are the deployer's responsibility.
- **Not proof that events are true or complete.** The ledger records what the harness observed and chose to record (normalized outcomes and, on Target A, response digests). It does not show that the target behaved as recorded or that nothing was omitted before recording. Timestamps come from the harness host's clock.
- **Not the target's ledger.** For Target A only the target's own ledger-valid flag was read (74/74), which the harness did not recompute. The Target A harness ledger was written by an earlier ledger version (non-cryptographic chain hash); Target A was not re-run, so none of the SHA-256 claims on this page apply to it.

7. OWASP Top 10 for LLM Applications (2025) mapping

“Control relevant to this risk” means a tested control bears on the risk. It does not mean the whole OWASP category was assessed.

OWASP ITEM	CONTROLS RELEVANT TO THIS RISK	CATEGORY ASSESSED?	EVIDENCE IN THIS SAMPLE
LLM01 Prompt Injection	promptfoo `prompt-injection` strategy (stub target only)	No	Pipeline smoke test only; no claim about any real model.
LLM02 Sensitive Information Disclosure	Test 3 (post-revocation access via the tested tool); promptfoo `pii:direct`, `harmful:privacy` (stub only)	No	Limited evidence about post-revocation access through the one tool tested on Target A (0/31). Not a pass for the category. Stub: no claim.
LLM06 Excessive Agency	Tests 3, 4, 5, 6 (revocation, fail-closed, credential reuse, approval gates); promptfoo `excessive-agency` (stub)	Partly: credential and approval controls only	F-01 to F-04 on Target B. On Target A, tests 3, 5 and 6 passed within the scope tested. Tool-permission scoping and agent behavior were not assessed.
LLM10 Unbounded Consumption	Test 7 (rate limit)	No	F-05 on Target B; not applicable to Target A. Model-level resource consumption was not assessed.
Supporting: auditability	Test 8 (decision reasons, ledger)	Not mapped	F-06 on Target B and I-01 on Target A are auditability observations, not mapped to an OWASP category.
LLM03, 04, 05, 07, 08, 09	Not in scope for this sample	No	none

8. Caveats

- Target A was exercised only through its local test adapter, which uses a simulated clock. The latency numbers are local loopback wall-clock times, not a production SLA.
- On Target A, "replay" means revoked-credential reuse: presenting the revoked credential, and re-presenting the unlock operation, after revocation. The adapter can't present a pre-revocation credential *after* re-authorization, so that variant wasn't tested there (it was on Target B).
- The adapter has no "stale approval" input. The approval matrix covers missing, insufficient, missing-second-factor and valid inputs; stale and mismatched approvals were covered on Target B.
- Target B is a deliberately vulnerable mock. Its findings show what the harness detects, not flaws in any real product.
- Timing: “0” enforcement delay means no post-revocation call was accepted in the window, sampled about every 100 ms; it is not a measured zero.
- Target B tests have no in-test positive controls; the clean configuration serves as the control (Section 3). The 74/74 ledger result on Target A is the target’s own flag. The harness ledger is tamper-evident, not tamper-proof: it is unsigned unless a key is supplied, detects removal of trailing entries only against an expected head or count, and is not proof that recorded events are true or complete (Section 6).

Appendix A: Commands

```
# Target A: start adapter on loopback with a fresh in-memory 48-char token, run mapped tests, stop
cd agent-harness
# writes report.md, results.json and evidence-ledger.* to <profile-out>
npm run target:reference
npm run target:reference:adapter-test       # adapter's own test: ok 1, pass 1, fail 0

# Target B: clean + 6 flawed configurations, expected-failure matrix
# 8/8 PASS (7 configurations + 1 signed run); HARNESS_SELFTEST_EXIT_CODE=0
npm run selftest
npm run typecheck                           # tsc --noEmit, clean
npm run test:ledger                         # ledger unit checks (also run by selftest)

# Prompt-level smoke (local stub only)
cd promptfoo && npm run redteam:smoke:generate && npm run redteam:smoke:eval   # Oct 20 pipeline
smoke test: 18/18 expected stub responses; real-model security not evaluated

# Ledger verification
# Head and count are read from the sidecar next to the file, or pass --expected-head / --expected-
count.
# If LEDGER_SIGNING_KEY is set, the HMAC signature is required to verify; the key is never printed.
npm run verify-ledger -- selftest-out/bad-slow-revoke/evidence-ledger.jsonl --tamper-demo
# expect: VALID; edit, delete, swap detected; tail truncation detected only against an expected
head/count
for f in selftest-out/*/evidence-ledger.jsonl; do npm run -s verify-ledger -- "$f"; done
# expect: all VALID, head matched sidecar
```